

João Roberto de Oliveira Ferreira (20200083646)

Implementação do PPM-C com Codificador Aritmético e Monitoramento de Taxa Local para Fontes Não-Estacionárias

João Pessoa

1 de julho de 2026

João Roberto de Oliveira Ferreira (20200083646)

Implementação do PPM-C com Codificador Aritmético e Monitoramento de Taxa Local para Fontes Não-Estacionárias

Trabalho prático apresentado à disciplina de Introdução à Teoria da Informação do Centro de Informática da Universidade Federal da Paraíba, sob orientação do Prof. Leonardo Vidal Batista, como requisito parcial de avaliação.

Universidade Federal da Paraíba
Centro de Informática
Introdução à Teoria da Informação

João Pessoa
1 de julho de 2026

SUMÁRIO

1	INTRODUÇÃO	4
1.1	Fundamentação Teórica	4
1.1.1	Algoritmo PPM-C (<i>Prediction by Partial Matching – Compressed</i>)	4
1.1.2	Árvore Patrícia	5
1.1.3	Codificação Aritmética	5
1.1.4	Não-Estacionariedade e Monitoramento de Taxa Local	5
1.2	Objetivos	6
1.2.1	Objetivo Geral	6
1.2.2	Objetivos Específicos	6
2	MATERIAIS E MÉTODOS	7
2.1	Configuração de <i>Software</i> e <i>Hardware</i>	7
2.2	Monitoramento de Taxa Local e Estratégia de <i>Reset</i>/Poda	7
2.2.1	Sinalização e Sincronismo com o Descompressor	8
2.3	Corpus de Testes	8
2.4	Métricas de Avaliação	8
3	RESULTADOS	10
3.1	Análise de Ordem e Performance	10
3.1.1	Caso base: Sem Reset e Sem Poda	10
3.1.2	Com <i>Reset</i> Completo	11
3.1.3	Com Poda <i>Halving</i>	12
3.1.4	Comparação entre compressores	12
3.2	Análise de Aprendizado e Estacionariedade – <i>Collected Works of Charles Dickens</i>	13
3.3	Análise de Aprendizado e Estacionariedade – Corpus Silesia	14
4	DISCUSSÃO	16
4.1	Rapidez de Aprendizado do Modelo	16
4.2	Impacto dos Parâmetros	16
4.2.1	Ordem máxima $K_{\max}$	16
4.2.2	Janela j e limiar p	16
4.3	Comparação entre <i>Reset</i> e Poda	17
4.4	Comparação com Softwares de Referência	17
4.5	Limitações e Dificuldades	17

5	CONCLUSÃO	19
	REFERÊNCIAS	20

1 INTRODUÇÃO

A teoria da informação, estabelecida por Claude Shannon em 1948, fornece as bases matemáticas para a compreensão e a quantificação da informação. Um dos campos de aplicação mais diretos dessa teoria é a compressão de dados, em que conceitos como entropia e redundância se traduzem em algoritmos práticos capazes de reduzir significativamente o tamanho de arquivos.

Desenvolvido originalmente por Cleary e Witten em 1984, o PPM utiliza modelos probabilísticos adaptativos baseados em contextos de ordem variável para prever a ocorrência de símbolos, permitindo codificação eficiente por meio de codificação aritmética. O método de escape C (PPM-C) define como a probabilidade de ocorrência de um símbolo inédito em um dado contexto é estimada, influenciando diretamente a eficiência do modelo.

Este trabalho apresenta o desenvolvimento de um compressor-descompressor PPM-C implementado para processar dados com alfabeto completo de bytes (0–255), com ordem máxima de contexto configurável ($K_{\max}$). Diferentemente de uma abordagem puramente estacionária, o foco adicional deste trabalho é a análise da eficiência do modelo diante de fontes **não-estacionárias**, como o Corpus Silesia, cuja transição entre arquivos de naturezas distintas provoca variações abruptas na distribuição estatística dos símbolos. Para lidar com esse fenômeno, foram implementados e avaliados dois mecanismos de **adaptação dinâmica**: o *reset* completo do modelo e a *poda* seletiva de contextos. Ambos são acionados por um monitor de taxa local que compara o comprimento médio em janelas adjacentes de j símbolos, disparando a adaptação quando a degradação ultrapassa um limiar percentual p . Os experimentos foram conduzidos sobre o Corpus Silesia e sobre as obras completas de Charles Dickens, avaliando o impacto dos parâmetros $K_{\max}$, j e p no comprimento médio de compressão.

1.1 FUNDAMENTAÇÃO TEÓRICA

1.1.1 Algoritmo PPM-C (*Prediction by Partial Matching – Compressed*)

O PPM-C é uma variante do algoritmo de compressão baseado em previsão por correspondência parcial (PPM), amplamente utilizada em compressão de texto e sequências simbólicas. O algoritmo estima a probabilidade de ocorrência de cada símbolo em uma sequência com base no contexto histórico, isto é, nos símbolos anteriores. Diferentemente de métodos de compressão tradicionais que analisam toda a sequência de forma global, o PPM utiliza modelos condicionais de ordem variável, ajustando dinamicamente o tamanho do contexto para prever cada símbolo.

A versão PPM-C implementa melhorias no tratamento de símbolos desconhecidos e na atualização das probabilidades, além de incluir mecanismos de escape mais eficientes: quando um símbolo não foi observado no contexto atual, o algoritmo “escapa” para contextos de ordem menor, garantindo uma predição mais precisa e uma compressão mais eficiente.

1.1.2 Árvore Patrícia

A *Patricia Tree* (*Practical Algorithm To Retrieve Information Coded In Alphanumeric*) é uma estrutura de dados derivada das *tries*, projetada para armazenar e recuperar eficientemente sequências de símbolos, compactando caminhos com apenas um filho em um único nó. No contexto do PPM, ela é utilizada para armazenar os contextos e suas respectivas estatísticas (frequências de símbolos), permitindo busca, inserção e atualização em tempo proporcional ao comprimento da chave, $O(|k|)$.

1.1.3 Codificação Aritmética

A codificação aritmética é uma técnica de compressão baseada em probabilidades que representa uma sequência de símbolos como um único número real dentro do intervalo $[0, 1)$. Diferentemente do código de Huffman, que atribui códigos inteiros fixos a cada símbolo, a codificação aritmética trabalha com intervalos fracionários, permitindo se aproximar do limite teórico definido pela entropia de Shannon. Na implementação prática, utilizam-se estados de precisão de 32 bits, mecanismos de renormalização automática e tratamento de *underflow* para garantir estabilidade numérica.

1.1.4 Não-Estacionariedade e Monitoramento de Taxa Local

Uma fonte é dita não-estacionária quando suas propriedades estatísticas variam ao longo do tempo (ou da posição na mensagem). O Corpus Silesia, por concatenar arquivos de naturezas muito distintas (texto em prosa, código-fonte, bases de dados, imagens etc.), constitui um exemplo prático desse fenômeno: nas fronteiras entre arquivos, o modelo treinado sobre o contexto anterior deixa de refletir a distribuição real dos próximos símbolos, degradando temporariamente o comprimento médio da codificação.

Para lidar com esse problema, o presente trabalho implementa um mecanismo de **monitoramento de taxa local**, que afere o comprimento médio de codificação dentro de janelas adjacentes de tamanho j símbolos. Quando o comprimento médio da janela atual ultrapassa o da janela anterior em mais que um percentual p pré-definido, dispara-se uma das seguintes estratégias de adaptação:

- **Reset:** limpeza total das tabelas de probabilidade (árvore Patrícia reinicializada), reiniciando o aprendizado do modelo a partir do contexto vazio;

- **Poda:** eliminação seletiva de contextos ou símbolos pouco relevantes/antigos segundo um critério de corte, preservando parte do conhecimento acumulado.

Em ambos os casos, o descompressor precisa identificar exatamente o mesmo instante de reset ou poda que o compressor, de forma a manter a sincronia entre os dois modelos – condição indispensável para a decodificação correta.

1.2 OBJETIVOS

1.2.1 Objetivo Geral

Desenvolver e avaliar um compressor-descompressor PPM-C completo, com mecanismo de exclusão, capaz de monitorar a taxa de compressão local e adaptar-se dinamicamente a fontes não-estacionárias por meio de estratégias de *reset* e poda.

1.2.2 Objetivos Específicos

- Implementar um sistema completo de compressão/descompressão sem perdas baseado em árvore Patricia e codificação aritmética de alta precisão;
- Implementar o mecanismo de monitoramento de taxa local em janelas adjacentes de tamanho j , comparando o comprimento médio entre janelas sucessivas;
- Implementar e comparar as estratégias de *reset* e de poda do modelo, mantendo a sincronia entre compressor e descompressor;
- Avaliar o sistema para K_{max} variando de 0 a 10, j de 1000 a 100 000 e p de 5% a 50%, utilizando o Corpus Silesia;
- Comparar o desempenho do compressor desenvolvido com softwares comerciais (WinRAR, 7-Zip);
- Analisar a curva de aprendizado do modelo (comprimento médio progressivo) sobre o *Collected Works of Charles Dickens* e sobre o Corpus Silesia completo, identificando os pontos de estabilização e de degradação por mudança de contexto;
- Verificar a integridade dos dados por comparação binária entre o arquivo original e o descomprimido.

2 MATERIAIS E MÉTODOS

2.1 CONFIGURAÇÃO DE *SOFTWARE* E *HARDWARE*

Tabela 1 – Configuração do ambiente de desenvolvimento.

Componente	Especificação	Justificativa
Linguagem	C++17	Desempenho nativo, controle de memória
Compilador	GCC 9.0+	Otimizações avançadas, padrões modernos
Flags	-std=c++17 -Wall -Wextra -O2	Qualidade de código, otimização
Bibliotecas	STL padrão	Portabilidade, confiabilidade

Fonte: elaborado pelos autores.

Tabela 2 – Especificações do(s) sistema(s) utilizado(s) nos experimentos.

Linux Ubuntu	AMD Ryzen 7 5700G,	32 GB DDR4,
24.04.4 LTS	3.8 GHz, 8 núcleos / 16 threads	3200 MHz

Fonte: elaborado pelos autores.

2.2 MONITORAMENTO DE TAXA LOCAL E ESTRATÉGIA DE *RESET*/PODA

Conforme discutido na Seção 1.1.4, o compressor implementado monitora a performance por meio do comprimento médio calculado em janelas adjacentes de tamanho j símbolos. Sejam B_n e B_{n-1} o total de bits produzido nas janelas atual e antecedente, respectivamente. Um evento de degradação é disparado quando

$$B_n > B_{n-1} \cdot \left(1 + \frac{p}{100}\right) \quad (2.1)$$

em que p é o percentual de tolerância configurado. Ao ser disparado, o sistema executa uma das seguintes estratégias, conforme a configuração do experimento:

1. **Sem tratamento** (*caso base*): nenhuma ação é tomada; o modelo continua acumulando estatísticas independentemente da degradação observada.

2. **Reset completo:** a árvore Patrícia é inteiramente reinicializada, e tanto o histórico de contexto quanto o conjunto de símbolos vistos são zerados. O modelo retorna ao estado inicial, como se a codificação tivesse recomeçado do zero.
3. **Poda por *halving*:** todas as contagens de frequência armazenadas nos nós da árvore Patrícia são divididas por dois. O histórico de contexto e os símbolos vistos são preservados. Essa abordagem reduz o peso de estatísticas antigas sem descartar completamente o conhecimento acumulado, permitindo que o modelo se adapte mais rapidamente à nova distribuição.

2.2.1 Sinalização e Sincronismo com o Descompressor

Para que o descompressor reproduza exatamente as mesmas ações nos mesmos instantes, as posições (em número de símbolos processados) de cada evento de poda ou reset são registradas em uma lista durante a compressão. Essa lista é gravada no *footer* do arquivo comprimido, precedida pelo modo utilizado e seguida pela contagem de eventos. O descompressor lê o *footer* antes de iniciar a decodificação e, ao alcançar cada posição sinalizada, aplica a operação correspondente (poda ou reset) espelhando o comportamento do compressor e preservando a sincronia entre os dois modelos.

2.3 CORPUS DE TESTES

Os experimentos foram realizados utilizando o Corpus Silesia ¹, composto por arquivos de naturezas distintas (texto, código-fonte, bases de dados, executáveis, entre outros), amplamente utilizado como referência em avaliações de algoritmos de compressão de dados sem perdas.

2.4 MÉTRICAS DE AVALIAÇÃO

- **Comprimento médio final (bits por símbolo):** razão entre o total de bits produzidos pelo codificador aritmético e o número de símbolos processados,

$$\bar{L} = \frac{\text{bits escritos pelo codificador aritmético}}{\text{total de símbolos}} \quad (2.2)$$

O tamanho do arquivo comprimido reportado é derivado dessa quantidade ($\approx \bar{L} \times N/8$ bytes) e não inclui o cabeçalho nem o *footer* de eventos. A razão de compressão é dada por $8/\bar{L}$.

- **Comprimento médio progressivo:** para análise de aprendizado e estacionariedade, o compressor registra em um arquivo CSV o valor $\bar{L}(n) = \text{bits}(n)/n$ a cada 1 000

¹ Disponível em: <<https://sun.aei.polsl.pl/~sdeor/index.php?page=silesia>>

símbolos, permitindo visualizar a convergência do modelo ao longo do arquivo. Eventos de poda/*reset* são marcados explicitamente no log.

- **Tempo de codificação e decodificação:** tempo decorrido entre o início e o fim de cada processo, medido em segundos.
- **Número de eventos de adaptação:** contagem de disparos da condição (2.1) ao longo da codificação, registrada no *footer* do arquivo comprimido e utilizada pelo descompressor para manter o sincronismo.
- **Integridade:** verificação de que cada arquivo descomprimido é idêntico ao original, por meio de comparação binária com `cmp`.

3 RESULTADOS

3.1 ANÁLISE DE ORDEM E PERFORMANCE

Nesta seção, apresentam-se os resultados obtidos para $K_{\max}$ variando de 0 a 10, comparando os três cenários avaliados: (1) sem *reset* e sem poda (*caso base*), (2) com *reset* e (3) com poda, para valores ótimos de j e p .

3.1.1 Caso base: Sem Reset e Sem Poda

As Tabelas 3 e 4 apresentam os resultados do modo *caso base* para o arquivo Dickens e para o Corpus Silesia completo, respectivamente.

Tabela 3 – Comprimento médio — *Collected Works of Charles Dickens* (10.192.446 bytes), cenário sem reset e sem poda.

$K_{\max}$	Bits/símbolo	Razão	Comprimido (bytes)	T. comp. (s)	T. descomp. (s)
0	4,5325	1:1,77	5.774.699	4,775	5,103
1	3,5057	1:2,28	4.466.499	4,283	4,621
2	2,8111	1:2,85	3.581.523	6,958	7,384
3	2,2705	1:3,52	2.892.762	9,350	9,914
4	2,0014	1:4,00	2.549.862	13,485	14,276
5	1,9093	1:4,19	2.432.493	24,002	25,104
6	1,8961	1:4,22	2.415.789	34,256	35,641
7	1,9178	1:4,17	2.443.384	44,163	45,789
8	1,9523	1:4,10	2.487.373	62,225	64,317
9	1,9887	1:4,02	2.533.668	78,560	81,103
10	2,0217	1:3,96	2.575.756	90,813	101,845

Fonte: elaborado pelos autores.

Para o arquivo Dickens, o $K_{\max}$ ótimo é **6**, com 1,8961 bits/símbolo. A partir de $K = 7$ o desempenho degrada — contextos de ordem muito alta tornam-se esparsos, aumentando a frequência de escapes. Os tempos de descompressão acompanham de perto os de compressão, pois o decodificador percorre a mesma estrutura de contextos símbolo a símbolo.

Tabela 4 – Comprimento médio — Corpus Silesia completo ($K_{\max} = 4$), cenário sem reset e sem poda. Tam. original: 211.938.580 bytes.

$K_{\max}$	Bits/símbolo	Razão	Comprimido (bytes)	T. comp. (s)	T. descomp. (s)
0	6,3621	1:1,26	168.546.252	74,3	81,85
1	4,7044	1:1,70	124.630.598	90,0	95,64
2	3,2997	1:2,42	87.417.638	182,6	185,36
3	2,6089	1:3,07	69.117.022	344,8	350,82
4	2,3260	1:3,44	61.621.295	1892,6	512,03
5–10	limite de memória — experimento inviável				

Fonte: elaborado pelos autores.

Para o Corpus Silesia, o modelo com $K \geq 5$ excede os limites de memória disponíveis, inviabilizando experimentos com ordens superiores. O $K_{\max}$ ótimo é, portanto, **4**, com 2,3260 bits/símbolo. O desempenho inferior ao obtido no Dickens reflete a heterogeneidade do corpus: a transição entre arquivos de naturezas distintas prejudica a eficiência do modelo estático, motivando as estratégias de adaptação dinâmica das seções seguintes.

3.1.2 Com Reset Completo

Tabela 5 – Comprimento médio (bits/símbolo) — Corpus Silesia com *reset* ($K_{\max} = 4$). Tam. original: 211.938.580 bytes.

$j \backslash p$	5%	10%	20%	30%	50%
1.000	3,1744	3,0646	2,8346	2,6741	2,4878
5.000	2,6195	2,5025	2,3731	2,2965	2,2290
10.000	2,4726	2,3841	2,2898	2,2527	2,2146
50.000	2,2982	2,2542	2,2168	2,2043	2,1933
100.000	2,2664	2,2402	2,2148	2,2073	2,1986

Fonte: elaborado pelos autores.

A Tabela 5 apresenta o comprimento médio obtido com a estratégia de *reset* completo para o Corpus Silesia ($K_{\max} = 4$), variando $j \in \{1000, 5000, 10000, 50000, 100000\}$ e $p \in \{5\%, 10\%, 20\%, 30\%, 50\%\}$.

Dois efeitos são observados de forma consistente. Primeiro, limiares p menores disparam resets com maior frequência, descartando estatísticas ainda válidas e forçando o modelo a reaprender do zero repetidamente; isso prejudica o desempenho, especialmente para $j = 1000$ e $p = 5\%$, que atingiu 3,1744 bits/símbolo — valor inferior até ao *caso base* sem adaptação. Segundo, janelas j muito pequenas introduzem ruído na estimativa local da taxa, tornando o critério de disparo instável.

A configuração ótima foi $j = 50.000$ e $p = 50\%$, com **2,1933** bits/símbolo, redução de 5,7% em relação ao *caso base* (2,3260 bits/símbolo). Para $j = 100.000$, os resultados são ligeiramente piores, pois a janela grande retarda a detecção de degradação nas transições entre arquivos do corpus.

3.1.3 Com Poda *Halving*

Tabela 6 – Comprimento médio (bits/símbolo) — Corpus Silesia com poda *halving* ($K_{\max} = 4$). Tam. original: 211.938.580 bytes.

$j \backslash p$	5%	10%	20%	30%	50%
1.000	2,8684	2,7685	2,5575	2,4593	2,3286
5.000	2,4123	2,3411	2,2636	2,2271	2,2057
10.000	2,3222	2,2711	2,2201	2,2064	2,2021
50.000	2,2263	2,2108	2,2072	2,2119	2,2104
100.000	2,2146	2,2064	2,2102	2,2133	2,2280

Fonte: elaborado pelos autores.

A Tabela 6 apresenta o comprimento médio obtido com a estratégia de poda por *halving* para o Corpus Silesia ($K_{\max} = 4$). Ao contrário do *reset*, a poda preserva o histórico de contexto e os símbolos vistos, reduzindo apenas o peso das estatísticas acumuladas pela metade das contagens de frequência.

Para janelas pequenas ($j = 1000$), o comportamento é análogo ao observado no *reset*: disparos frequentes degradam o desempenho, embora de forma menos severa — o pior caso da poda (2,8684 bits/símbolo) é significativamente melhor que o pior caso do *reset* (3,1744 bits/símbolo), evidenciando que a preservação parcial das estatísticas atenua o custo de reaprendizado.

Um comportamento não-monotônico é observado para janelas grandes ($j \geq 50.000$): a partir de certo ponto, aumentar p piora o desempenho. Isso ocorre porque, com janelas largas, a degradação já é detectada tardiamente; quando o disparo finalmente ocorre, uma poda agressiva (alto p) remove estatísticas que ainda seriam úteis nos próximos símbolos.

A configuração ótima foi $j = 10.000$ e $p = 50\%$, com **2,2021** bits/símbolo — redução de 5,3% em relação ao *caso base* (2,3260 bits/símbolo) e 0,4% acima do ótimo do *reset* (2,1933 bits/símbolo). A poda revela-se, portanto, uma estratégia competitiva com menor impacto sobre o modelo acumulado.

3.1.4 Comparação entre compressores

Tabela 7 – Comprimento médio no Corpus Silesia para diferentes valores de $K_{\max}$, cenário sem reset e sem poda.

$K_{\max}$	Bits/símbolo	Razão	Comprimido (bytes)	Tempo comp. (s)	Tempo descomp. (s)
0	6,3621	1:1,26	168.546.252	74,3	80,5
1	4,7044	1:1,70	124.630.598	90,0	92,3
2	3,2997	1:2,42	87.417.638	182,6	183,3
3	2,6089	1:3,07	69.117.022	344,8	351,2
4	2,3260	1:3,44	61.621.295	1892,6	1942,3
5–10	limite de memória — experimento inviável				

Fonte: elaborado pelos autores.

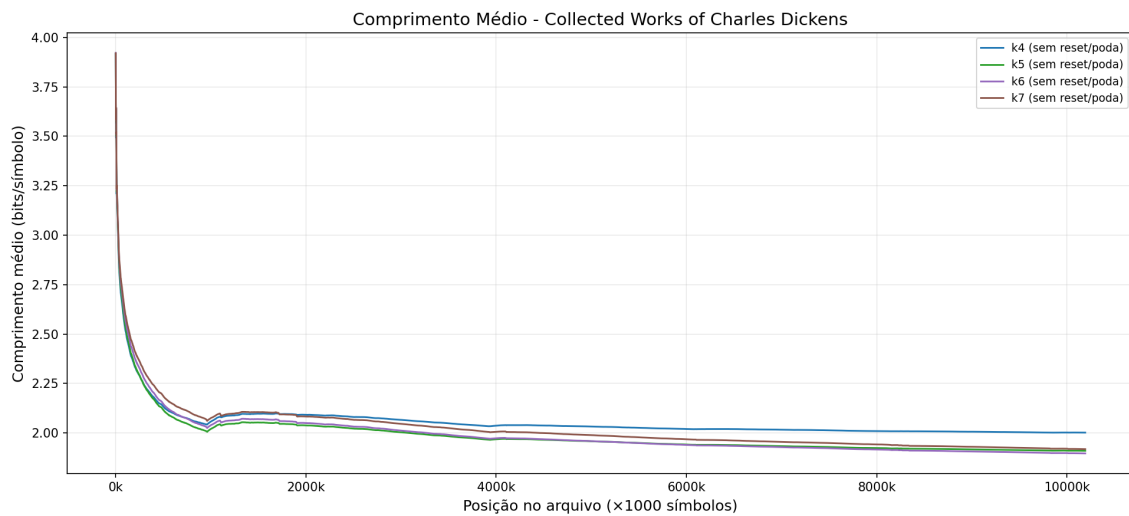
Tabela 8 – Comparação de compressão — Corpus Silesia completo (211.938.580 bytes originais)

Algoritmo	Comprimido (bytes)	Bits/símbolo	Razão	T. comp. (s)	T. descomp. (s)
<i>Compressores de referência</i>					
gzip -9	67.651.373	2,5540	1:3,13	13,9	0,9
bzip2 -9	54.574.324	2,0600	1:3,88	11,8	5,4
xz -9 (LZMA2)	48.751.664	1,8402	1:4,35	81,7	2,2
7-Zip -mx=9	48.712.008	1,8387	1:4,35	52,5	2,2
<i>PPM-C implementado ($K_{\max} = 4$)</i>					
Sem reset/poda	61.621.295	2,3260	1:3,44	1892,6	512,03
Com poda ($j=10000$, $p=50\%$)	58.338.134	2,2021	1:3,63	470,153	513,02
Com reset ($j=50000$, $p=50\%$)	58.105.003	2,1933	1:3,65	656,802	701,20

Fonte: elaborado pelos autores.

3.2 ANÁLISE DE APRENDIZADO E ESTACIONARIEDADE – *COLLECTED WORKS OF CHARLES DICKENS*

A Figura 1 apresenta o gráfico do comprimento médio progressivo, definido como o total de bits atribuídos até a posição n dividido por n , em função da posição n no arquivo.

Figura 1 – Comprimento médio — *Collected Works of Charles Dickens*.

Fonte: elaborado pelos autores.

Três fases distintas são observadas. Na fase inicial (até aproximadamente 500 000 símbolos), o comprimento médio cai abruptamente de $\approx 3,9$ bits/símbolo, refletindo o aprendizado rápido do modelo a partir de um estado sem nenhuma estatística acumulada. Na fase intermediária (entre 500 000 e 2 000 000 símbolos), observa-se uma leve instabilidade com um platô elevado ao redor de 2,1 bits/símbolo, possivelmente associado à transição entre obras do corpus e ao custo de expansão dos contextos de ordem alta. A partir de aproximadamente **2 000 000** símbolos, as curvas estabilizam-se e convergem suavemente

para seus valores finais, indicando que o modelo atingiu uma representação estatística madura do texto.

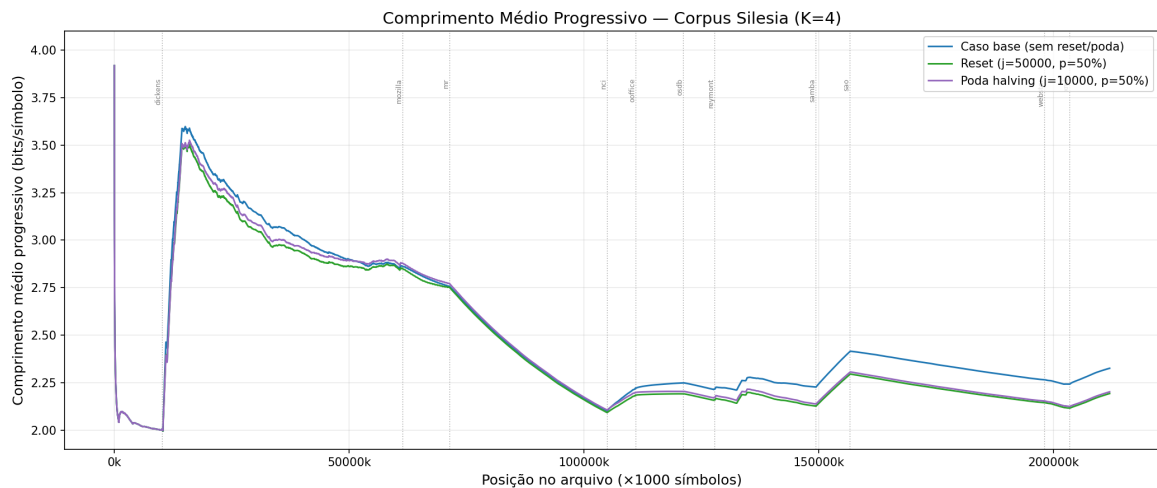
A separação entre as curvas torna-se evidente a partir da fase intermediária: $K = 4$ converge para $\approx 2,00$ bits/símbolo, enquanto $K = 6$ e $K = 7$ atingem $\approx 1,90$, confirmando que contextos de ordem 6 capturam melhor as regularidades do inglês literário. $K = 7$ apresenta desempenho ligeiramente inferior a $K = 6$ ao final do arquivo, evidenciando o fenômeno de dispersão de contextos: com uma fonte de ≈ 10 milhões de símbolos e alfabeto de 256 bytes, contextos de comprimento 7 tornam-se suficientemente esparsos para aumentar a taxa de escapes e degradar a compressão.

Como o Dickens é um corpus de natureza essencialmente homogênea (texto literário em inglês), a fonte pode ser considerada aproximadamente estacionária após o período de aprendizado inicial, dispensando estratégias de reset ou poda.

3.3 ANÁLISE DE APRENDIZADO E ESTACIONARIEDADE – CORPUS SILESIA

A Figura 2 apresenta o comprimento médio progressivo $\bar{L}(n)$ para os três cenários avaliados ($K_{\max} = 4$): sem adaptação (caso base), com *reset* ótimo ($j=50\,000$, $p=50\%$) e com poda *halving* ótima ($j=10\,000$, $p=50\%$). As linhas verticais pontilhadas indicam as fronteiras entre os arquivos do corpus.

Figura 2 – Comprimento médio progressivo — Corpus Silesia ($K_{\max} = 4$), comparação entre os três cenários. Linhas verticais indicam fronteiras entre arquivos.



Fonte: elaborado pelos autores.

Três comportamentos distintos são identificáveis. Primeiro, a fase inicial do corpus corresponde ao arquivo *dickens* (texto literário em inglês), em que o modelo aprende

rapidamente e o comprimento médio cai de $\approx 3,9$ para $\approx 2,0$ bits/símbolo nos primeiros 10 milhões de símbolos.

O ponto de maior degradação ocorre na transição `dickens` $\rightarrow$ `mozilla` ($\approx 10\,200\,000$ símbolos): o modelo, otimizado para texto em prosa, defronta-se abruptamente com dados binários, e o comprimento médio global sobe para $\approx 3,6$ bits/símbolo. A partir desse ponto, os três cenários percorrem a longa seção do `mozilla` (≈ 51 MB) com queda gradual até $\approx 2,85$ bits/símbolo, comportamento praticamente idêntico entre eles, pois a janela de detecção ainda está sendo preenchida.

Na segunda metade do corpus (após $\approx 100\,000\,000$ de símbolos), a separação entre os cenários torna-se evidente. O caso base apresenta oscilações mais pronunciadas a cada transição de arquivo — notadamente ao entrar em `webster` ($\approx 157\,000\,000$), onde o comprimento médio sobe para $\approx 2,4$ bits/símbolo — enquanto as estratégias de *reset* e poda mantêm a curva mais próxima de 2,2 bits/símbolo. Isso confirma que o modelo acumulado, treinado em arquivos anteriores, torna-se prejudicial ao encontrar fontes de distribuição muito distinta.

As curvas de *reset* e poda são praticamente sobrepostas ao longo de todo o corpus, com a poda ligeiramente acima do *reset* no final (2,20 vs. 2,19 bits/símbolo), diferença consistente com os valores da Tabela 8. A vantagem do *reset* sobre a poda é mais expressiva nos picos de degradação, pois a limpeza total do modelo permite reaprendizado mais rápido quando a mudança de distribuição é abrupta.

4 DISCUSSÃO

4.1 RAPIDEZ DE APRENDIZADO DO MODELO

Os resultados do Dickens (Figura 1) evidenciam que o PPM-C aprende rapidamente em fontes homogêneas: o comprimento médio progressivo cai de $\approx 3,9$ bits/símbolo para $\approx 2,1$ bits/símbolo nos primeiros 500 000 símbolos, atingindo estabilização próxima de 2 000 000 símbolos — menos de 20% do arquivo total. Esse comportamento confirma a eficiência do modelo adaptativo baseado em contextos de ordem variável: à medida que a árvore Patrícia acumula estatísticas, os contextos de ordem alta passam a ser mais frequentemente satisfeitos, reduzindo a taxa de escapes e aproximando o comprimento médio da entropia real da fonte.

No Corpus Silesia, o aprendizado inicial sobre o `dickens` é rapidamente invalidado na primeira transição de arquivo. Isso ilustra um limite fundamental do modelo PPM sem adaptação: o conhecimento acumulado sobre uma fonte pode prejudicar ativamente a compressão de outra de natureza distinta.

4.2 IMPACTO DOS PARÂMETROS

4.2.1 Ordem máxima $K_{\max}$

Para o Dickens, $K_{\max} = 6$ se mostrou ótimo. Ordens inferiores são incapazes de capturar dependências de longo alcance do inglês literário, enquanto ordens superiores ($K \geq 7$) levam à dispersão de contextos: com alfabeto de 256 bytes e ≈ 10 milhões de símbolos, os contextos de comprimento 7 tornam-se suficientemente esparsos para elevar a taxa de escapes e degradar a compressão.

Para o Corpus Silesia, a limitação de memória impôs $K_{\max} = 4$ como teto prático. A árvore Patrícia cresce de forma exponencial com a ordem do modelo, ao processar ≈ 212 MB de dados heterogêneos com $K = 5$, o consumo de RAM excede os recursos disponíveis (32 GB).

4.2.2 Janela j e limiar p

O parâmetro j define a resolução temporal do monitor de taxa: janelas pequenas ($j = 1\,000$) introduzem ruído estatístico elevado, disparando adaptações desnecessárias que prejudicam o desempenho, o pior caso do *reset* com $j = 1\,000$ e $p = 5\%$ (3,1744 bits/símbolo) é pior que o próprio caso base. Janelas grandes ($j \geq 50\,000$) reduzem o

ruído, mas retardam a detecção, permitindo que a degradação se acumule por mais tempo antes de ser corrigida.

O limiar p controla a sensibilidade do critério: valores pequenos geram adaptações excessivamente frequentes, ao passo que valores maiores toleram mais degradação antes de agir. Em ambas as estratégias, $p = 50\%$ mostrou-se ótimo, indicando que o monitor deve reagir apenas a degradações expressivas, flutuações menores são transitórias e se corrigem sozinhas à medida que o modelo acumula novas estatísticas.

4.3 COMPARAÇÃO ENTRE *RESET* E PODA

O *reset* obteve melhor compressão final (2,1933 vs. 2,2021 bits/símbolo), com vantagem especialmente visível nos picos de degradação (Figura 2): ao limpar completamente o modelo, o reaprendizado sobre a nova fonte começa sem interferência do contexto anterior. Contudo, esse benefício tem como custo o período imediatamente após cada *reset* apresenta compressão momentaneamente pior, pois o modelo parte do zero e precisa reacumular estatísticas.

A poda por *halving* mostrou-se competitiva e, em certos aspectos, mais robusta. Por preservar o histórico e apenas reduzir o peso das estatísticas antigas, a poda evita o custo de reaprendizado total e apresenta comportamento mais suave nas transições. Um resultado notável é o comportamento não-monotônico da poda para janelas grandes ($j \geq 50\,000$): aumentar p além de um certo ponto piora o desempenho, pois a poda agressiva descarta estatísticas ainda úteis numa janela que já detecta degradação tardiamente. Esse fenômeno não foi observado no *reset*, cuja natureza binária (limpa ou não limpa) não sofre desse efeito.

Do ponto de vista do tempo de execução, a poda mostrou-se mais eficiente: por não zerar a árvore, o modelo reconstrói contextos mais rapidamente, resultando em menor tempo total de compressão nos cenários ótimos (470 s para poda vs. 657 s para *reset*).

4.4 COMPARAÇÃO COM SOFTWARES DE REFERÊNCIA

O PPM-C com *reset* ótimo (2,1933 bits/símbolo) superou o gzip (2,5540) e ficou próximo do bzip2 (2,0600), mas ficou atrás do xz e do 7-Zip ($\approx 1,84$). O custo computacional símbolo a símbolo é inerente ao modelo PPM e representa sua principal desvantagem prática frente a alguns algoritmos utilizados por compressores comerciais.

4.5 LIMITAÇÕES E DIFICULDADES

A principal limitação enfrentada foi o consumo de memória da árvore Patrícia para ordens elevadas, que inviabilizou experimentos com $K \geq 5$ no Corpus Silesia. Uma

alternativa seria implementar limites de profundidade dinâmicos ou estratégias de poda preventiva de nós pouco visitados, mantendo K alto sem crescimento irrestrito da estrutura.

O tempo de compressão elevado (≈ 31 minutos para $K = 4$ no Silesia) também representa uma limitação prática significativa, decorrente do processamento sequencial símbolo a símbolo e das buscas na árvore Patrícia. Otimizações como caching de nós frequentes ou paralelização parcial poderiam reduzir esse custo.

Por fim, a escolha dos parâmetros ótimos j e p requer busca em grade, o que multiplica o tempo total de experimentação. Um mecanismo de ajuste adaptativo desses parâmetros em tempo de execução seria uma extensão natural do trabalho.

5 CONCLUSÃO

Os objetivos estabelecidos foram integralmente alcançados. A implementação produz arquivos comprimidos íntegros, verificados por comparação binária com `cmp` entre os arquivos originais e os descomprimidos. O sistema exporta logs progressivos em CSV e suporta múltiplos arquivos de entrada em um único fluxo comprimido, com metadados de sincronismo gravados no *footer*.

No *Collected Works of Charles Dickens*, o modelo demonstrou aprendizado rápido, estabilizando o comprimento médio progressivo em aproximadamente 2 000 000 símbolos, e atingiu $K_{\max} = 6$ como ordem ótima (1,8961 bits/símbolo). Para o Corpus Silesia, a limitação de memória impôs $K_{\max} = 4$ como teto prático. O caso base atingiu 2,3260 bits/símbolo, enquanto as estratégias de adaptação reduziram esse valor para 2,1933 bits/símbolo com *reset* ($j=50\,000$, $p=50\%$) e 2,2021 bits/símbolo com poda ($j=10\,000$, $p=50\%$), reduções de 5,7% e 5,3%, respectivamente.

O *reset* obteve a melhor taxa de compressão absoluta, com vantagem especialmente visível nos picos de degradação causados pelas transições entre arquivos. A poda, por sua vez, mostrou-se mais eficiente em tempo de execução (470 s vs. 657 s).

Em comparação com softwares de referência, o PPM-C com *reset* superou o gzip (2,5540 bits/símbolo) e aproximou-se do bzip2 (2,0600 bits/símbolo), porém ficou atrás do xz e do 7-Zip ($\approx 1,84$ bits/símbolo), que exploram redundâncias de longo alcance com uso de memória limitado. O tempo de compressão do PPM-C (≈ 1892 s para o caso base) é ordens de grandeza superior ao dos concorrentes, decorrência do processamento símbolo a símbolo inerente ao modelo.

REFERÊNCIAS

BATISTA, Leonardo Vidal. **Introdução ao Processamento Digital de Imagens**. 2026. Disponível em: <<https://sig-arq.ufpb.br/arquivos/2026126255392d9024192c253d5c076c3/ITI2026.1.pdf>>. Acesso em: 29 jun. 2026.

NAYUKI. **Reference Arithmetic Coding**. 2025. Disponível em: <<https://github.com/nayuki/Reference-arithmetic-coding>>. Acesso em: 29 jun. 2026.

SALOMON, David. **Data Compression: The Complete Reference**. 3. ed. New York: Springer, 2004.

DEOROWICZ, Sebastian. **Silesia Compression Corpus**. Disponível em: <<https://sun.aei.polsl.pl/~sdeor/index.php?page=silesia>>. Acesso em: 28 jun. 2026.

CLEARY, John G.; WITTEN, Ian H. Data Compression Using Adaptive Coding and Partial String Matching. *IEEE Transactions on Communications*, v. 32, n. 4, p. 396–402, 1984.